

## **ПОДГОТОВКА К ЗАЩИТЕ**

Контрольные вопросы к лабораторной работе № 1

Выполнил: Миронин Картюем Дмитриевич  
Группа: П 3431

# Подготовка к защите лабораторной работы № 1

Документ содержит короткий ответ, который удобно произнести, и пояснение на случай дополнительного вопроса преподавателя.

## 1. Почему bcrypt предпочтительнее SHA-256 для паролей?

**Короткий ответ.** SHA-256 спроектирован как очень быстрый алгоритм. Это удобно для проверки целостности, но помогает злоумышленнику перебирать миллиарды вариантов пароля. bcrypt специально медленный, автоматически использует соль и имеет настраиваемый cost factor, поэтому массовый перебор намного дороже.

**Подробнее.** Хэш-функция пароля должна быть односторонней, уникально солить каждый пароль и замедлять подбор. Одна только соль не делает SHA-256 медленным: она защищает от заранее вычисленных rainbow tables, но не от перебора конкретной утечки. У bcrypt соль хранится внутри строки хэша вместе с параметром стоимости. В проекте используется cost 12. Пароль проверяется функцией bcrypt.compare, а обратное восстановление пароля невозможно. Современная альтернатива — Argon2id, которая дополнительно требует много памяти.

## 2. В чём разница между SAST и DAST?

**Короткий ответ.** SAST анализирует исходный код без запуска приложения и показывает потенциально опасные конструкции. DAST тестирует уже работающее приложение снаружи, отправляя запросы как клиент или атакующий.

**Подробнее.** SAST удобно запускать рано при каждом commit: он обычно указывает файл и строку, но может давать ложные срабатывания и не видит реальную конфигурацию среды. DAST не требует исходников и замечает ошибки поведения развёрнутой системы, но хуже объясняет место дефекта и не покрывает недостижимые ветви. Подходы дополняют друг друга. В работе ESLint Security — SAST. npm audit не DAST, а SCA: проверка состава сторонних компонентов.

## 3. Как работает JWT и как сервер проверяет его подлинность?

**Короткий ответ.** JWT состоит из Base64URL-кодированных header, payload и криптографической подписи. После входа сервер помещает в payload идентификатор пользователя и срок действия, подписывает токен секретом. При каждом запросе сервер заново вычисляет и проверяет подпись и проверяет exp. Изменённый или просроченный токен отклоняется.

**Подробнее.** Header содержит тип и алгоритм, payload — claims, например sub, login, iat, exp. Payload лишь кодируется, но не шифруется, поэтому секреты туда класть нельзя. В проекте применяется HS256: один секрет служит для подписи и проверки. Разрешённый алгоритм задан явно, чтобы токен не мог переключить проверку на неожиданный алгоритм. JWT передаётся как Authorization: Bearer ....

Выход пользователя сам по себе не отзывает уже выданный JWT; для досрочного отзыва нужны короткий TTL, список отозванных токенов или схема access/refresh.

## 4. Чем опасен отказ от аудита зависимостей?

**Короткий ответ. Даже безопасный собственный код использует сторонние пакеты.**

В старой версии библиотеки может быть известная CVE, вредоносный код или уязвимая транзитивная зависимость. Без SCA команда может долго не знать, что готовый путь атаки уже опубликован.

**Подробнее. Риски включают выполнение произвольного кода, утечки, DoS,** компрометацию цепочки поставки и проблемы лицензирования. Lock-файл обеспечивает воспроизводимость, но фиксирует и уязвимую версию. Поэтому `npm ci` и `npm audit --audit-level=high` запускаются в CI, а обновление должно сопровождаться тестами. Результат аудита надо анализировать: не каждое предупреждение достижимо, но критические проблемы нельзя просто скрывать.

## 5. Как именно проект защищён от SQL-инъекции?

В репозиториях используются prepared statements и placeholders ?. SQL-команда компилируется отдельно, а пользовательское значение передаётся драйверу как данные. Поэтому кавычки и SQL-операторы внутри логина не становятся частью команды. Экранирование HTML не защищает от SQLi — это разные контексты.

## 6. Как именно проект защищён от XSS?

API отдаёт JSON и не вставляет данные в HTML. На выходе пользовательские строки дополнительно проходят HTML-экранирование, поэтому специальные символы не могут образовать тег или обработчик события при последующем отображении. Helmet добавляет защитные заголовки. На реальном фронтенде всё равно нужно использовать безопасный text binding и не передавать строку в innerHTML.

## 7. Почему пароль не шифруют, а хэшируют?

Шифрование обратимо: сервер с ключом может получить исходный пароль, а утечка ключа раскрывает все записи. Для проверки пароля исходное значение не нужно: достаточно сравнить его с односторонним адаптивным хэшем. Поэтому применяют bcrypt/Argon2, а шифрование используют для данных, которые необходимо потом прочитать.

## 8. Зачем JWT имеет срок действия?

Украденный бессрочный токен давал бы доступ постоянно. Claim exp ограничивает окно атаки. Чем короче срок, тем ниже риск, но тем чаще нужна повторная аутентификация или refresh token. В лабораторной по умолчанию установлен один час, в тестах — пять минут.

## 9. Почему автор поста берётся из JWT, а не из тела запроса?

Тело полностью контролирует клиент. Если принять authorId от клиента, он может создать запись от имени другого пользователя — это Broken Access Control. После проверки подписи сервер доверяет sub токена и использует его как автора.

## 10. Для чего нужны rate limit, ограничение JSON и Helmet?

Rate limit замедляет перебор паролей. Лимит тела снижает риск расходования памяти огромным запросом. Helmet централизованно выставляет набор безопасных HTTP-заголовков. Это дополнительные уровни защиты, а не замена проверке входа, JWT и правильной работе с данными.

## 11. Что проверяет pipeline?

При каждом push и pull request GitHub Actions устанавливает точные зависимости через npm ci, запускает семь интеграционных тестов, SAST через ESLint Security и SCA через npm audit. Job имеет только read-доступ к содержимому репозитория и ограничение времени десять минут.

## 12. Где в проекте проявляются KISS, DRY и ООП?

KISS — маленькое приложение без лишнего ORM и сложной иерархии. DRY — один JWT middleware защищает все /api/\*, общие функции выполняют валидацию и экранирование. ООП — Database, репозитории и AuthService инкапсулируют свои обязанности; зависимости приходят в конструкторы. Применена композиция. Делать наследование только ради демонстрации было бы усложнением и нарушило бы KISS.

## Мини-чеклист перед защитой

1. Уметь показать успешный login и скопировать JWT.
2. Показать HTTP 401 для /api/data без токена.
3. Создать пост со строкой <script> и показать экранированный ответ.
4. Показать prepared statements в репозиториях.
5. Показать bcrypt-хэш в БД и отсутствие открытого пароля.
6. Открыть успешный GitHub Actions run с тестами, SAST и SCA.







